

ArcVault Intake and Triage Pipeline: Architecture Write-Up

What I built

An automated support triage pipeline built in n8n. It receives incoming messages via webhook, sends them to GPT-4o for classification and enrichment, applies deterministic routing rules, and writes structured records to Google Sheets. Flagged messages go to a separate escalation queue for human review. The whole pipeline, from spec to working implementation, took about two hours.

Stack and reasoning

n8n: Already experienced with it and self-host on a VPS, so setup was fast. Clean workflow JSON export and built-in retry handling made it the obvious choice.

Google Sheets: Practical output layer any team can open and act on immediately. Easy to read during a demo. Swapping it for a database later is a small change.

Webhook trigger: Most control, closest to a real production setup. Fires instantly on each message, easy to test from any REST client.

GPT-4o: Fast, cheap, and reliable on structured tasks. Native JSON mode guarantees a parseable response every time. Initially tested GPT-4o-mini but it misclassified the 403 login error as an Incident/Outage, triggering a false escalation. GPT-4o handled all five cases correctly.

Raw HTTP Request node: More transparent than the OpenAI wrapper. Every parameter is explicit and visible, which made debugging faster and gives full control over the request.

How it works

Each message is normalized on arrival, stamped with an ID and timestamp. It then goes to GPT-4o in a single call that returns category, priority, confidence, extracted entities, and a plain-English summary. A Code node validates and sanitizes the output. A second Code node applies the routing and escalation rules deterministically. The result is written to the Triage_Output sheet, and escalated records get a copy in the Escalation_Queue sheet. The webhook responds with the full JSON record.

The model classifies; n8n decides the destination. Keeping routing logic out of the prompt means rules are auditable and can be changed without touching the LLM.

Routing and escalation

Category maps to queue as follows:

- Bug Report and Incident/Outage go to Engineering
- Feature Request goes to Product
- Billing Issue goes to Billing
- Technical Question goes to IT/Security

Escalation overrides routing under three conditions:

- Confidence below 0.70 — model is not certain enough to auto-route
- Incident signal detected in the message or category, regardless of confidence

- Billing discrepancy above \$500, calculated from extracted amounts or a stated overcharge

The billing rule applies to the discrepancy magnitude, not the invoice total. Sample 3 has a \$260 discrepancy against a \$1,240 invoice and correctly does not escalate. A stated overcharge like "you charged me \$800 extra" is also handled via a dedicated entity field, so the rule fires even when only one dollar figure appears in the message.

Escalated records preserve the intended queue in a `queue_attempted` field so reviewers know where the message would have gone.

What I would change

Two AI calls: Splitting classification and summary into separate calls avoids quality tradeoffs. Slightly higher cost but cleaner output and easier to tune each prompt independently.

Feedback workflow: Corrections made by human reviewers currently go nowhere. A feedback loop that captures approved and overridden records as labelled examples would let the system improve over time, and eventually support fine-tuning a smaller, faster, cheaper model on real data.

Dead-letter handling: If parsing fails or the Sheets append errors, the message is currently lost. A dead-letter tab with retry logic would make the pipeline production-safe.